# Temporal Tutorial

Let's make this fun, practical, and easy to follow. Imagine we're building a cool system together, like an app to manage a pizza delivery service, and we'll use Temporal to handle all the tricky timing and workflow stuff. Ready? Let's dive in!

## What's Temporal Workflow, and Why Should You Care?

Temporal is a framework for managing **complex, long-running workflows**. Think of it as a conductor orchestrating a sequence of steps:

- **Workflows**: Define the process (e.g., "cook, then deliver").
- **Activities**: Perform individual tasks (e.g., "bake the pizza").
- **Reliability**: Temporal ensures workflows resume from where they left off after failures.

💡 **Use Case:** Imagine handling thousands of pizza orders—some might get delayed, fail, or need retrying. Temporal ensures these workflows **complete successfully**, even across crashes and failures.

### Why Is It Awesome?

Imagine you're running a pizza shop. Orders come in, pizzas get cooked, and drivers deliver them. But what if the oven crashes or the driver gets stuck in traffic? Temporal is like your smart assistant that keeps everything on track, no matter what happens.

**Why Use It?** For our pizza app, Temporal ensures every order finishes, even if things go wrong—like a late delivery or a power outage.

## Setting Up Your Temporal Playground

First, let's get Temporal running locally and set up a Golang project.

### What You'll Need:

- **Golang**: Install it from golang.org if you haven't.
- **Temporal Server**: We'll run it locally using Docker (easiest way).

### 1️⃣ Install Dependencies

Option 1: Using Docker (Recommended)

```
docker run --rm -p 7233:7233 temporalio/server:latest
```

This runs a Temporal server locally on localhost:7233.

Option 2: Using Temporal CLI (For macOS/Linux Users)

```
brew install temporal
```

Start the server:

```
temporal server start-dev
```

### 2️⃣ Create a New Golang Project

```
mkdir pizza-temporal && cd pizza-temporal
```

Initialize a Go module:

```
go mod init pizza-temporal
```

Install the Temporal SDK:

```
go get go.temporal.io/sdk@latest
```

## The Basics – Workflows and Activities

Let's write code to handle a pizza order. We'll break it into two parts: the **workflow** (the plan) and **activities** (the jobs):

1. **Workflow**: The big plan—like "make and deliver a pizza." It's durable and keeps going even if your app restarts.
2. **Activity**: The actual work—like "cook the pizza" or "drive to the customer." These are the steps inside a workflow.

### First workflow

workflow.go

```go
package main

import (
	"time"
	"go.temporal.io/sdk/workflow"
)

// PizzaOrderWorkflow is the plan for a pizza order
func PizzaOrderWorkflow(ctx workflow.Context, orderID string) (string, error) {
	options := workflow.ActivityOptions{
		StartToCloseTimeout: 5 * time.Minute, // 5 minutes max per job
	}
	ctx = workflow.WithActivityOptions(ctx, options)
```

```
	// Step 1: Cook
	var cookResult string
	err := workflow.ExecuteActivity(ctx, CookPizzaActivity, orderID).Get(ctx,
&cookResult)
	if err != nil {
		return "", err
	}

	// Step 2: Deliver
	var deliverResult string
	err = workflow.ExecuteActivity(ctx, DeliverPizzaActivity, orderID).Get(ctx,
&deliverResult)
	if err != nil {
		return "", err
	}

	return "Order " + orderID + " done: " + cookResult + " and " + deliverResult,
nil
}
```

**Activities (activity.go)**

```
package main

import (
	"context"
	"time"
)

// CookPizzaActivity pretends to cook
func CookPizzaActivity(ctx context.Context, orderID string) (string, error) {
	time.Sleep(2 * time.Second) // Fake cooking
	return "Pizza cooked for " + orderID, nil
}

// DeliverPizzaActivity pretends to deliver
func DeliverPizzaActivity(ctx context.Context, orderID string) (string, error) {
	time.Sleep(3 * time.Second) // Fake delivery
	return "Pizza delivered for " + orderID, nil
}
```

**Worker and Client (main.go) - Start Everything**

```
package main

import (
	"context"
```

```
	"log"
	"go.temporal.io/sdk/client"
	"go.temporal.io/sdk/worker"
)

func main() {
	// Connect to Temporal
	c, err := client.Dial(client.Options{})
	if err != nil {
		log.Fatalln("Can't connect:", err)
	}
	defer c.Close()

	// Set up a worker
	w := worker.New(c, "pizza-task-queue", worker.Options{})
	w.RegisterWorkflow(PizzaOrderWorkflow)
	w.RegisterActivity(CookPizzaActivity)
	w.RegisterActivity(DeliverPizzaActivity)

	// Start worker in the background
	go func() {
		err := w.Run(worker.InterruptCh())
		if err != nil {
			log.Fatalln("Worker failed:", err)
		}
	}()

	// Start an order
	wf, err := c.ExecuteWorkflow(
		context.Background(),
		client.StartWorkflowOptions{
			ID:        "pizza-order-1",
			TaskQueue: "pizza-task-queue",
		},
		PizzaOrderWorkflow,
		"pizza-123",
	)
	if err != nil {
		log.Fatalln("Workflow failed:", err)
	}

	// Get the result
	var result string
	err = wf.Get(context.Background(), &result)
	if err != nil {
		log.Fatalln("No result:", err)
	}
	log.Println(result)
}
```

**Run the Application**

1. Temporal server is already running if not? Run this command:

```
temporal server start-dev.
```

2. Run the code:

```
go run .
```

3. Expected output:

```
Order pizza-123 completed: Pizza cooked for pizza-123 and Pizza delivered for
pizza-123
```

## Core Concepts

**Workflows Keep Going**: If your app crashes, Temporal saves the plan and picks up where it stopped.

**Activities Do Stuff**: They're the hands-on tasks—like calling a database or turning on the oven.

**Task Queues**: Think of "pizza-task-queue" as a to-do list for workers. You can have many queues for different jobs!

### Retries and Timeouts

Let's add some cool features to our pizza app.

Add Retries (If Something Fails) in ActivityOptions, before running activity:

What if the oven breaks? Tell Temporal to try again:

```
options := workflow.ActivityOptions{
      StartToCloseTimeout: 5 * time.Minute,
      RetryPolicy: &temporal.RetryPolicy{
            InitialInterval:    1 * time.Second, // Wait 1 sec
            BackoffCoefficient: 2.0,             // Double wait each retry
            MaximumAttempts:    3,               // Try 3 times
      },
}
```

### Idempotency

Ensure activities can be safely retried without side effects (e.g., avoid duplicate charges).

# Intermediate Features

## Signals: Handling External Input

What if a customer changes their order? Use signals to update in runtime

Update **PizzaOrderWorkflow** in workflow.go

```
func PizzaOrderWorkflow(ctx workflow.Context, orderID string) (string, error) {
	options := workflow.ActivityOptions{StartToCloseTimeout: 5 * time.Minute}
	ctx = workflow.WithActivityOptions(ctx, options)

	updateChan := workflow.GetSignalChannel(ctx, "update-order")
	var cookResult string
	err := workflow.ExecuteActivity(ctx, CookPizzaActivity, orderID).Get(ctx,
&cookResult)
	if err != nil {
		return "", err
	}

	var newOrderID string
	select {
	case updateChan.Receive(ctx, &newOrderID):
		orderID = newOrderID // New order!
	default:
	}

	var deliverResult string
	err = workflow.ExecuteActivity(ctx, DeliverPizzaActivity, orderID).Get(ctx,
&deliverResult)
	if err != nil {
		return "", err
	}
	return "Order " + orderID + " done: " + cookResult + " and " + deliverResult,
nil
}
```

Add to main.go after ExecuteWorkflow:

```
go func() {
	time.Sleep(3 * time.Second)
	err := c.SignalWorkflow(context.Background(), "pizza-order-1", "",
"update-order", "pizza-456")
	if err != nil {
		log.Println("Signal failed:", err)
	}
}()
```

### Timers: Set Deadlines

**Theory**: Stop if delivery is too slow.

**Snippet**: In workflow.go, add before delivery:

```
timer := workflow.NewTimer(ctx, 10*time.Second)
var deliveryDone bool
err = workflow.ExecuteActivity(ctx, DeliverPizzaActivity, orderID).Get(ctx,
&deliverResult)
if err == nil {
      deliveryDone = true
}
workflow.Select(ctx,
      workflow.Case(timer, func(ctx workflow.Context) bool {
            return !deliveryDone // Too late!
      }),
)
if !deliveryDone {
      return "Delivery too slow for " + orderID, nil
}
```

## Advanced Features

### Handle Multiple Orders

**For busy nights, use Child Workflows:**

**Theory**: Handle multiple orders with mini-plans (child workflows).
**Snippet**: Add to workflow.go:

```
func MultiPizzaWorkflow(ctx workflow.Context, orderIDs []string) (string, error) {
      var results []string
      for _, id := range orderIDs {
            childCtx := workflow.WithChildOptions(ctx,
workflow.ChildWorkflowOptions{})
            future := workflow.ExecuteChildWorkflow(childCtx, PizzaOrderWorkflow, id)
            var result string
            if err := future.Get(ctx, &result); err != nil {
                  return "", err
            }
            results = append(results, result)
      }
      return strings.Join(results, "; "), nil
}
```

Update main.go:

```
w.RegisterWorkflow(MultiPizzaWorkflow)
// Replace ExecuteWorkflow with:
wf, err := c.ExecuteWorkflow(
      context.Background(),
      client.StartWorkflowOptions{
            ID:        "multi-pizza-1",
            TaskQueue: "pizza-task-queue",
      },
      MultiPizzaWorkflow,
      []string{"pizza-123", "pizza-456"},
)
```

### Example for Child Workflows:

Use child workflows when a parent workflow needs to delegate work to independent sub-processes. For example, if a pizza shop receives multiple orders, each order can run as a child workflow while the parent tracks overall progress.

## Error Handling and Compensation

**Theory**: Undo steps if something fails.

**Snippet**: Update PizzaOrderWorkflow in workflow.go:

```
func PizzaOrderWorkflow(ctx workflow.Context, orderID string) (string, error) {
      options := workflow.ActivityOptions{StartToCloseTimeout: 5 * time.Minute}
      ctx = workflow.WithActivityOptions(ctx, options)

      var cookResult string
      err := workflow.ExecuteActivity(ctx, CookPizzaActivity, orderID).Get(ctx,
&cookResult)
      if err != nil {
            return "", err
      }

      var deliverResult string
      err = workflow.ExecuteActivity(ctx, DeliverPizzaActivity, orderID).Get(ctx,
&deliverResult)
      if err != nil {
            workflow.ExecuteActivity(ctx, UndoCookPizzaActivity, orderID)
            return "", err
      }
      return "Order " + orderID + " done: " + cookResult + " and " + deliverResult,
nil
}
```

```
// Add this
func UndoCookPizzaActivity(ctx context.Context, orderID string) (string, error) {
	return "Cooking undone for " + orderID, nil
}
```

In main.go, add:

```
w.RegisterActivity(UndoCookPizzaActivity)
```

## Workflow Execution & Lifecycle in Temporal

Workflows in Temporal go through various lifecycle stages, from creation to completion. Let's break it down step by step.

---

### 🔹 Workflow Lifecycle Stages

A Temporal Workflow has these key states:

1️⃣ **Pending** → Workflow is created but not yet started.
2️⃣ **Running** → Workflow is executing.
3️⃣ **Completed** → Workflow finished successfully.
4️⃣ **Failed** → Workflow encountered an error and failed.
5️⃣ **Timed Out** → Workflow exceeded its execution time and was stopped.
6️⃣ **Canceled** → Workflow was manually canceled.
7️⃣ **Terminated** → Workflow was forcefully stopped.
8️⃣ **Continued-As-New** → Workflow created a new instance to avoid long histories.

Let's understand workflow States in Golang.

**Define the Workflow (`workflow.go`)**

```
package main

import (
	"fmt"
	"time"

	"go.temporal.io/sdk/workflow"
)

// OrderWorkflow simulates an order process
func OrderWorkflow(ctx workflow.Context, orderID string) (string, error) {
	logger := workflow.GetLogger(ctx)
	logger.Info("Workflow started", "OrderID", orderID)
```

```
	// Set activity options
	options := workflow.ActivityOptions{
		StartToCloseTimeout: 10 * time.Second,
	}
	ctx = workflow.WithActivityOptions(ctx, options)

	// Execute an activity (order processing)
	var result string
	err := workflow.ExecuteActivity(ctx, ProcessOrderActivity, orderID).Get(ctx,
&result)
	if err != nil {
		logger.Error("Workflow failed", "Error", err)
		return "", err // ❌ Workflow will move to FAILED state
	}

	logger.Info("Workflow completed", "OrderID", orderID)
	return result, nil // ✅ Workflow will move to COMPLETED state
}
```

**Define the Activity (`activity.go`)**

```
package main

import (
	"context"
	"fmt"
	"time"
)

// ProcessOrderActivity simulates order processing
func ProcessOrderActivity(ctx context.Context, orderID string) (string, error) {
	fmt.Println("Processing order:", orderID)
	time.Sleep(2 * time.Second) // Simulate processing time

	// Simulate an error for testing failure cases
	if orderID == "order-fail" {
		return "", fmt.Errorf("order processing failed")
	}

	return fmt.Sprintf("Order %s processed successfully", orderID), nil
}
```

**Start the Workflow & Handle Failures (`main.go`)**

```
package main

import (
	"context"
	"fmt"
```

```
	"log"

	"go.temporal.io/sdk/client"
	"go.temporal.io/sdk/worker"
)

func main() {
	// Connect to Temporal Server
	c, err := client.Dial(client.Options{})
	if err != nil {
		log.Fatalln("Unable to connect to Temporal:", err)
	}
	defer c.Close()

	// Create a worker to handle workflows & activities
	w := worker.New(c, "order-task-queue", worker.Options{})
	w.RegisterWorkflow(OrderWorkflow)
	w.RegisterActivity(ProcessOrderActivity)

	// Start the worker in the background
	go func() {
		err := w.Run(worker.InterruptCh())
		if err != nil {
			log.Fatalln("Worker failed:", err)
		}
	}()

	// Start a workflow (Normal order)
	wf1, err := c.ExecuteWorkflow(
		context.Background(),
		client.StartWorkflowOptions{
			ID:        "order-1",
			TaskQueue: "order-task-queue",
		},
		OrderWorkflow,
		"order-123",
	)
	if err != nil {
		log.Fatalln("Workflow failed to start:", err)
	}

	// Start a workflow that will fail
	wf2, err := c.ExecuteWorkflow(
		context.Background(),
		client.StartWorkflowOptions{
			ID:        "order-2",
			TaskQueue: "order-task-queue",
		},
		OrderWorkflow,
		"order-fail",
```

```
	)
	if err != nil {
		log.Fatalln("Workflow failed to start:", err)
	}

	// Get workflow results
	var result1, result2 string
	wf1.Get(context.Background(), &result1)
	wf2.Get(context.Background(), &result2)

	fmt.Println("Result 1:", result1)
	fmt.Println("Result 2:", result2) // This will show an error due to failure
}
```

## Handling Workflow Failures

**When a workflow fails, we can retry it, cancel it, or handle errors gracefully.**

### ✅ How to Retry a Failed Workflow?

**Modify `ActivityOptions` inside `OrderWorkflow`:**

```
options := workflow.ActivityOptions{
    StartToCloseTimeout: 10 * time.Second,
    RetryPolicy: &temporal.RetryPolicy{
        InitialInterval:    2 * time.Second, // First retry after 2s
        BackoffCoefficient: 2.0,             // Doubles the wait time
        MaximumAttempts:    3,               // Maximum 3 attempts
    },
}
ctx = workflow.WithActivityOptions(ctx, options)
```

**What happens here?**

- If the activity fails, Temporal automatically retries it 3 times.
- The wait time doubles after each failure (2s → 4s → 8s).

### Manually Cancel or Terminate a Workflow

If you need to cancel or forcefully stop a workflow, you can do this:

### ✅ Cancel a Running Workflow

```
err := c.CancelWorkflow(context.Background(), "order-1")
if err != nil {
    log.Println("Cancel failed:", err)
} else {
    log.Println("Workflow order-1 canceled")
}
```

👉 This moves the workflow to CANCELED state.

✅ **Forcefully Terminate a Workflow**

```
err := c.TerminateWorkflow(context.Background(), "order-1", "Order no longer needed")
if err != nil {
    log.Println("Termination failed:", err)
} else {
    log.Println("Workflow order-1 terminated")
}
```

👉 This moves the workflow to TERMINATED state.

## 📌 Summary

| State | When It Happens? | How to Handle It? |
|---|---|---|
| **Running** | Workflow is executing | Wait for completion |
| **Completed** | Workflow finishes successfully | Check results |
| **Failed** | Error occurred in execution | Use retry policies or handle errors |
| **Timed Out** | Workflow exceeded allowed time | Increase timeout or optimize logic |
| **Canceled** | Manually canceled by the user | Use `CancelWorkflow()` |
| **Terminated** | Forcefully stopped | Use `TerminateWorkflow()` |
| **Continued-As-New** | Workflow restarted to avoid long history | Use `workflow.ContinueAsNew()` |

# Workflow Versioning in Temporal

### Why Do We Need Workflow Versioning?

In Temporal, workflows persist state. But what happens if you change your workflow code while old workflows are still running?

- **Without versioning** → Old workflows break when they resume after deployment.
- **With versioning** → Old and new workflows coexist smoothly without failures.

### What Happens If You Change a Running Workflow?

Imagine this workflow code is already running in production:

```
func OrderWorkflow(ctx workflow.Context, orderID string) (string, error) {
    err := workflow.ExecuteActivity(ctx, ProcessOrderActivity, orderID).Get(ctx, nil)
    if err != nil {
        return "", err
    }
    return "Order processed", nil
}
```

**Now, you add a new activity (`NotifyCustomerActivity`):**

```
func OrderWorkflow(ctx workflow.Context, orderID string) (string, error) {
    err := workflow.ExecuteActivity(ctx, ProcessOrderActivity, orderID).Get(ctx, nil)
    if err != nil {
        return "", err
    }

    // 🚀 New activity added!
    err = workflow.ExecuteActivity(ctx, NotifyCustomerActivity, orderID).Get(ctx,
nil)
    if err != nil {
        return "", err
    }

    return "Order processed and customer notified", nil
}
```

**Problem?** ☠️

- Old workflows resume execution, but they don't know about NotifyCustomerActivity!
- This causes panic and failure.

## How Does Temporal Handle Versioning?

To ensure smooth transitions, Temporal provides:
✅ **workflow.GetVersion()** → Detects whether a workflow is running an old or new version.
✅ **Backward compatibility** → Allows old workflows to complete without breaking.

## How to Implement Workflow Versioning?

✅ **Solution:** Use workflow.GetVersion() to check if the new logic should run.

🔹 **Step 1: Add Versioning in Workflow**

**Modify your workflow:**

```
const VersionNotifyCustomer = 1 // Increase this number when making changes

func OrderWorkflow(ctx workflow.Context, orderID string) (string, error) {
    err := workflow.ExecuteActivity(ctx, ProcessOrderActivity, orderID).Get(ctx, nil)
    if err != nil {
        return "", err
    }

    // 🆕 Versioning logic
    version := workflow.GetVersion(ctx, "NotifyCustomerActivity",
workflow.DefaultVersion, VersionNotifyCustomer)
    if version >= VersionNotifyCustomer {
```

```
        err = workflow.ExecuteActivity(ctx, NotifyCustomerActivity, orderID).Get(ctx,
nil)
        if err != nil {
            return "", err
        }
    }

    return "Order processed", nil
}
```

🔹 **Step 2: Deploy the Change**

1️⃣ Deploy the new workflow code with versioning (VersionNotifyCustomer = 1).
2️⃣ Old workflows (started before the update) skip the new activity.
3️⃣ New workflows (started after the update) execute the new activity.

### 🔷 How Does `workflow.GetVersion()` Work?

| Workflow State | What Happens? |
|---|---|
| Workflow started before code update | `workflow.GetVersion()` returns `workflow.DefaultVersion` (skips new logic). |
| Workflow started after code update | `workflow.GetVersion()` returns `VersionNotifyCustomer = 1` (runs new logic). |
| Future updates | Just increase the version number (`VersionNotifyCustomer = 2`). |

**What If You Need Another Change Later?**

Suppose we now add another feature: SendInvoiceActivity.
1️⃣ Increase the version number to 2.
2️⃣ Add another workflow.GetVersion() check.

```
const VersionNotifyCustomer = 1
const VersionSendInvoice = 2 // 🚀 New version added

func OrderWorkflow(ctx workflow.Context, orderID string) (string, error) {
    err := workflow.ExecuteActivity(ctx, ProcessOrderActivity, orderID).Get(ctx, nil)
    if err != nil {
        return "", err
    }

    // 🚀 First version change
    version1 := workflow.GetVersion(ctx, "NotifyCustomerActivity",
workflow.DefaultVersion, VersionNotifyCustomer)
    if version1 >= VersionNotifyCustomer {
        err = workflow.ExecuteActivity(ctx, NotifyCustomerActivity, orderID).Get(ctx,
nil)
```

```
        if err != nil {
            return "", err
        }
    }

    // 🚀 Second version change
    version2 := workflow.GetVersion(ctx, "SendInvoiceActivity",
workflow.DefaultVersion, VersionSendInvoice)
    if version2 >= VersionSendInvoice {
        err = workflow.ExecuteActivity(ctx, SendInvoiceActivity, orderID).Get(ctx,
nil)
        if err != nil {
            return "", err
        }
    }

    return "Order processed", nil
}
```

**Key Benefits of Temporal Versioning**

✅ Old workflows don't break after a code update.
✅ New workflows run the latest logic without affecting old ones.
✅ You can safely make multiple workflow changes over time.

### 📌 Summary

| Scenario | Without Versioning ❌ | With Versioning ✅ |
|---|---|---|
| Add a new activity | Old workflows fail | Old workflows skip new logic |
| Remove an existing activity | Old workflows panic | Old workflows continue safely |
| Modify workflow execution order | Old workflows act unpredictably | Old workflows behave as expected |

# Querying Workflows in Temporal

### Why Do We Need Querying?

Workflows in Temporal can run for days, months, or even years. Sometimes, we need to:
✅ Check the status of a running workflow without modifying it.
✅ Retrieve real-time data from a workflow (e.g., order status in a delivery app).
✅ Monitor progress of long-running workflows.

👉 Solution: Temporal provides Queries to fetch workflow state without modifying execution.

## How Does Querying Work?

- Queries do not change workflow state (they are read-only).

- Queries are synchronous (they return a response immediately).
- You can query any active workflow using a client request.

### Example: Querying an Order Status in Golang

Let's build a pizza order workflow where we can:
✅ Start an order workflow.
✅ Check the real-time status of the order using queries.

#### Step 1: Define the Workflow (`workflow.go`)

```go
package main

import (
	"go.temporal.io/sdk/workflow"
)

// OrderWorkflow tracks the status of an order
func OrderWorkflow(ctx workflow.Context, orderID string) (string, error) {
	logger := workflow.GetLogger(ctx)
	logger.Info("Workflow started", "OrderID", orderID)

	// Workflow status variable (keeps track of progress)
	var orderStatus string

	// ✅ Register a query handler to return order status
	err := workflow.SetQueryHandler(ctx, "getStatus", func() (string, error) {
		return orderStatus, nil
	})
	if err != nil {
		return "", err
	}

	// Step 1: Processing order
	orderStatus = "Processing"
	workflow.Sleep(ctx, 3*workflow.Second) // Simulating work

	// Step 2: Cooking pizza
	orderStatus = "Cooking"
	workflow.Sleep(ctx, 5*workflow.Second) // Simulating work

	// Step 3: Out for delivery
	orderStatus = "Out for delivery"
	workflow.Sleep(ctx, 7*workflow.Second) // Simulating work

	// Step 4: Delivered
	orderStatus = "Delivered"
	return "Order " + orderID + " completed", nil
}
```

**How Does This Work?**

- orderStatus keeps track of the workflow progress.
- workflow.SetQueryHandler() allows clients to query "getStatus" anytime to get the current order status.

**Step 2: Register the Workflow in the Worker (main.go)**

```
package main

import (
	"context"
	"fmt"
	"log"

	"go.temporal.io/sdk/client"
	"go.temporal.io/sdk/worker"
)

func main() {
	// Connect to Temporal Server
	c, err := client.Dial(client.Options{})
	if err != nil {
		log.Fatalln("Unable to connect to Temporal:", err)
	}
	defer c.Close()

	// Create a worker
	w := worker.New(c, "order-task-queue", worker.Options{})
	w.RegisterWorkflow(OrderWorkflow)

	// Start the worker in the background
	go func() {
		err := w.Run(worker.InterruptCh())
		if err != nil {
			log.Fatalln("Worker failed:", err)
		}
	}()

	// ✅ Start a new workflow instance
	wf, err := c.ExecuteWorkflow(
		context.Background(),
		client.StartWorkflowOptions{
			ID:        "order-123",
			TaskQueue: "order-task-queue",
		},
		OrderWorkflow,
		"order-123",
	)
	if err != nil {
```

```
            log.Fatalln("Workflow failed to start:", err)
      }

      fmt.Println("Workflow started! Now, let's query its status...")

      // ✅ Query the workflow every 2 seconds
      for i := 0; i < 5; i++ {
            var status string
            err := c.QueryWorkflow(context.Background(), "order-123", "",
"getStatus").Get(&status)
            if err != nil {
                  log.Println("Query failed:", err)
            } else {
                  fmt.Println("Order Status:", status)
            }
      }
}
```

**Running the Query Example**

**1️⃣ Start the Temporal Server**

```
temporal server start-dev
```

**2️⃣ Run the Worker & Start Workflow**

```
go run main.go
```

**3️⃣ Expected Output:**

```
Workflow started! Now, let's query its status...
Order Status: Processing
Order Status: Cooking
Order Status: Out for delivery
Order Status: Delivered
```

✅ Even though the workflow is still running, we can check its status using queries!

## 🔷 When to Use Queries?

| Scenario | Use Queries? |
|---|---|
| Checking the real-time state of a workflow | ✅ Yes |
| Fetching order details in an e-commerce system | ✅ Yes |
| Tracking the progress of a long-running task | ✅ Yes |
| Modifying workflow state | ❌ No (Use Signals Instead) |

---

## 🔷 Queries vs. Signals

| Feature | Queries | Signals |
|---|---|---|
| **Purpose** | Fetch workflow data | Modify workflow data |
| **State changes?** | ❌ No | ✅ Yes |
| **Response type** | Synchronous (Instant) | Asynchronous |
| **Example Use Case** | Checking order status | Cancelling an order |

# Long-Running Workflows & Continue-As-New in Temporal

### Why Do We Need Continue-As-New?

Temporal Workflows store every event in history, including:
✅ Every activity execution
✅ Every timer
✅ Every signal/query

⚠️ Problem?

- If a workflow runs for months or years, its history grows too large, leading to performance issues.
- Temporal has a hard limit on history size (50,000 events per workflow).

👉 Solution? workflow.ContinueAsNew()

- Starts a new workflow execution but preserves workflow logic and carries forward data.
- Clears the old workflow history to prevent bloat.

Let's create a long-running order tracker that runs indefinitely by continuing as new every 24 hours.

## ✅ Step 1: Define the Workflow (`workflow.go`)

```
package main

import (
	"time"

	"go.temporal.io/sdk/workflow"
)

// LongRunningOrderWorkflow tracks orders continuously
func LongRunningOrderWorkflow(ctx workflow.Context, orderID string, runCount int)
error {
	logger := workflow.GetLogger(ctx)
	logger.Info("Workflow started", "OrderID", orderID, "RunCount", runCount)

	// Simulate order processing
	workflow.Sleep(ctx, 10*time.Second) // Simulate 10 sec work

	// ✅ After every run, continue-as-new to reset history
	if runCount < 3 { // Limit to 3 iterations for demo
		logger.Info("Continuing workflow as new...", "NextRun", runCount+1)
		return workflow.NewContinueAsNewError(ctx, LongRunningOrderWorkflow,
orderID, runCount+1)
	}

	logger.Info("Workflow completed!", "OrderID", orderID)
	return nil
}
```

## ✅ Step 2: Register & Start Workflow (`main.go`)

```
package main

import (
	"context"
	"fmt"
	"log"

	"go.temporal.io/sdk/client"
	"go.temporal.io/sdk/worker"
)

func main() {
	// Connect to Temporal Server
	c, err := client.Dial(client.Options{})
	if err != nil {
		log.Fatalln("Unable to connect to Temporal:", err)
	}
	defer c.Close()
```

```
	// Register workflow
	w := worker.New(c, "order-task-queue", worker.Options{})
	w.RegisterWorkflow(LongRunningOrderWorkflow)

	// Start worker
	go func() {
		err := w.Run(worker.InterruptCh())
		if err != nil {
			log.Fatalln("Worker failed:", err)
		}
	}()

	// ✅ Start the long-running workflow
	wf, err := c.ExecuteWorkflow(
		context.Background(),
		client.StartWorkflowOptions{
			ID:        "long-running-order",
			TaskQueue: "order-task-queue",
		},
		LongRunningOrderWorkflow,
		"order-123",
		1,
	)
	if err != nil {
		log.Fatalln("Workflow failed to start:", err)
	}

	// Wait for completion (should never complete in a real use case)
	fmt.Println("Workflow started:", wf.GetID())
}
```

**Running the workflow Example**

**1️⃣ Start the Temporal Server**

```
temporal server start-dev
```

**2️⃣ Run the Worker & Start Workflow**

```
go run main.go
```

**3️⃣ Expected Output:**

```
Workflow started (Run 1)
Continuing workflow as new... (Run 2)
Continuing workflow as new... (Run 3)
Workflow completed!
```

Each time ContinueAsNew is called, the history resets, preventing overload.

### 🔷 When to Use Continue-As-New?

| Scenario | Use Continue-As-New? |
|---|---|
| Workflow runs for months/years | ✅ Yes |
| Large workflow history size | ✅ Yes |
| Batch processing that resets daily | ✅ Yes |
| Short workflows (seconds/minutes) | ❌ No |

### What Happens Internally?

1️⃣ New execution starts immediately.
2️⃣ Old execution is marked as "Continued-As-New".
3️⃣ Workflow ID remains the same.
4️⃣ History is completely reset.

## Temporal Search Attributes – Querying Workflows by Business Data

#### Why Do We Need Search Attributes?

Temporal allows us to search and filter workflows based on business-specific data (e.g., order status, customer ID).

🔹 Without search attributes:

- You can only query workflows using Workflow ID or Run ID (which is limited).
- You can't search for all "Pending" orders or filter orders by customer ID.

🔹 With search attributes:
✅ Find all "pending" orders.
✅ Retrieve all orders for customer ID = 12345.
✅ Filter workflows by custom attributes like status, priority, etc.

### How Search Attributes Work?

1️⃣ Workflows store business-related metadata using workflow.UpsertSearchAttributes().
2️⃣ You can query workflows via the Temporal CLI, Web UI, or SDK.

Let's modify our **OrderWorkflow** to include search attributes.

#### ✅ Step 1: Define the Workflow with Search Attributes (workflow.go)

```
package main
```

```
import (
	"time"

	"go.temporal.io/sdk/workflow"
)

// OrderWorkflow processes an order and updates search attributes
func OrderWorkflow(ctx workflow.Context, orderID string, customerID int) (string,
error) {
	logger := workflow.GetLogger(ctx)
	logger.Info("Workflow started", "OrderID", orderID, "CustomerID", customerID)

	// ✅ Define search attributes (Metadata for filtering)
	attr := map[string]interface{}{
		"OrderID":    orderID,
		"CustomerID": customerID,
		"Status":     "Processing",
	}

	// ✅ Update search attributes
	err := workflow.UpsertSearchAttributes(ctx, attr)
	if err != nil {
		logger.Error("Failed to upsert search attributes", "Error", err)
		return "", err
	}

	// Simulate order processing
	workflow.Sleep(ctx, 5*time.Second) // Simulate processing

	// ✅ Update status to "Completed"
	attr["Status"] = "Completed"
	err = workflow.UpsertSearchAttributes(ctx, attr)
	if err != nil {
		logger.Error("Failed to update search attributes", "Error", err)
		return "", err
	}

	logger.Info("Order completed", "OrderID", orderID)
	return "Order processed successfully", nil
}
```

### ✅ Step 2: Register & Start Workflow (main.go)

```
package main

import (
	"context"
	"fmt"
	"log"
```

```
	"go.temporal.io/sdk/client"
	"go.temporal.io/sdk/worker"
)

func main() {
	// Connect to Temporal Server
	c, err := client.Dial(client.Options{})
	if err != nil {
		log.Fatalln("Unable to connect to Temporal:", err)
	}
	defer c.Close()

	// Register workflow
	w := worker.New(c, "order-task-queue", worker.Options{})
	w.RegisterWorkflow(OrderWorkflow)

	// Start worker
	go func() {
		err := w.Run(worker.InterruptCh())
		if err != nil {
			log.Fatalln("Worker failed:", err)
		}
	}()

	// ✅ Start workflow with search attributes
	wf, err := c.ExecuteWorkflow(
		context.Background(),
		client.StartWorkflowOptions{
			ID:        "order-456",
			TaskQueue: "order-task-queue",
			SearchAttributes: map[string]interface{}{
				"CustomerID": 12345,
				"Status":     "Pending",
			},
		},
		OrderWorkflow,
		"order-456",
		12345,
	)
	if err != nil {
		log.Fatalln("Workflow failed to start:", err)
	}

	fmt.Println("Workflow started:", wf.GetID())
}
```

## Searching Workflows Using CLI

After the workflow is running, you can **search for all workflows where CustomerID = 12345**:

```
temporal workflow list --query "CustomerID = 12345"
```

or find **all workflows where Status = "Processing"**:

```
temporal workflow list --query "Status = 'Processing'"
```

### 🔷 When to Use Search Attributes?

| Scenario | Use Search Attributes? |
|---|---|
| Filter orders by customer ID | ✅ Yes |
| Find all workflows in 'Pending' state | ✅ Yes |
| Find orders assigned to a specific employee | ✅ Yes |
| Modify workflow data | ❌ No (Use Signals Instead) |

# Monitoring & Observability in Temporal

## Why Do We Need Monitoring & Observability?

In production, we need to:
✅ Track running workflows & activities
✅ Identify failed workflows & retries
✅ Debug issues in long-running workflows
✅ Monitor system performance (latency, failures, retries, queue backlogs, etc.)

Temporal provides multiple observability tools, including:

- **Temporal Web UI** – Visual workflow tracking
- **CLI Commands** – Debug workflows via terminal
- **Logging** – Capture workflow & activity logs
- **Metrics & Tracing** – Monitor execution performance

## 1️⃣ Temporal Web UI – Visual Monitoring

The Temporal Web UI is the easiest way to view workflows, their states, and history.

### 1️⃣ Run Temporal Server

```
temporal server start-dev
```

### 2️⃣ Open Temporal Web UI

Go to: http://localhost:8233

Here, you can:

✅ View active & completed workflows

✅ Check workflow history & activity executions

✅ See workflow failures & retry attempts

### 🔹 2️⃣ Monitoring Workflows Using CLI

You can use the **Temporal CLI** to inspect workflows, check failures, and retry manually.

#### ✅ List all workflows

```
temporal workflow list
```

Shows all running workflows.

#### ✅ Check workflow details

```
temporal workflow describe --workflow-id order-123
```

Displays workflow state, execution history, and activity details.

#### ✅ View workflow history

```
temporal workflow show --workflow-id order-123
```

Shows each event in the workflow execution.

#### ✅ Retry a failed workflow

```
temporal workflow reset --workflow-id order-123 --reason "Fixing failure"
```

Restarts a failed workflow from a safe state.

### 🔹 3️⃣ Adding Logs to Workflows & Activities

Logs help debug workflows **inside Temporal Worker processes**.

#### ✅ Add Logging in Workflow (workflow.go)

```
package main

import (
	"time"
	"go.temporal.io/sdk/workflow"
)

// OrderWorkflow with logging
func OrderWorkflow(ctx workflow.Context, orderID string) (string, error) {
	logger := workflow.GetLogger(ctx)
	logger.Info("Workflow started", "OrderID", orderID)
```

```
	// Simulating processing
	workflow.Sleep(ctx, 5*time.Second)
	logger.Info("Processing completed", "OrderID", orderID)

	return "Order completed", nil
}
```

Logs appear in the Temporal Worker console.

✅ **Add Logging in Activity (activity.go)**

```
package main

import (
	"context"
	"log"
)

// ProcessOrderActivity with logging
func ProcessOrderActivity(ctx context.Context, orderID string) (string, error) {
	log.Println("Processing order:", orderID)
	return "Order processed", nil
}
```

Logs appear in the **Worker console**.

## 🔹 4️⃣ Enabling Metrics & Tracing

For advanced monitoring, Temporal supports Prometheus, Grafana, and OpenTelemetry.

✅ **Enable Prometheus Metrics**

1️⃣ Run Temporal with Prometheus enabled:

```
docker run -p 7233:7233 -p 9090:9090 temporalio/server:latest --prometheus
```

2️⃣ Open Prometheus UI at **http://localhost:9090**

3️⃣ Query Temporal metrics:

```
temporal_workflow_completed
temporal_activity_failed
temporal_task_queue_latency
```

✅ **Enable OpenTelemetry for Distributed Tracing**

To enable tracing with **Jaeger**:

1️⃣ Start Jaeger:

```
docker run -d --name jaeger -p 16686:16686 jaegertracing/all-in-one:latest
```

2️⃣ Open Jaeger UI at **http://localhost:16686**

3️⃣ View Temporal traces to analyze workflow execution paths.

## 🔷 Summary: Key Monitoring Methods

| Method | Purpose |
|---|---|
| Web UI | View workflow history & failures visually |
| CLI Commands | Debug workflows from terminal |
| Logging | Track workflow progress inside workers |
| Prometheus | Monitor Temporal performance metrics |
| Jaeger (Tracing) | Debug workflow execution flow |

**What Else Should You Know?**

- **Testing**: Temporal lets you test workflows easily—check the Temporal docs for how!
- **Versioning**: If you update your code, Temporal helps manage old and new workflows together.
- **Fixing Mistakes**: Add “undo” activities (like UndoCookPizzaActivity) if something fails mid-way.

**All Done!**

You’ve built a pizza app with Temporal! It’s crash-proof, retry-smart, and ready for busy nights. Play with it—add more features like “call the customer”!